

Manual Técnico — Vertice Muisca (VerticeMusicasWeb)

Campo	Valor
Proyecto	Sistema de gestión de autoservicio
Solución	Ingenieria softw.sln
Aplicación principal	VerticeMusicasweb/
Versión de framework	.NET 8.0
Base de datos	SQLite (vertice_musicas.db)
Fecha de referencia del manual	Mayo 2026
Tipo de arquitectura	Aplicación web monolítica (MVC). No es una arquitectura de microservicios.
Última actualización documentada	Entrada de inventario + informes (exportar/imprimir por sección)

Tabla de contenido

- [1. Introducción del sistema](#)
- [2. Objetivo general del proyecto](#)
- [3. Alcance del sistema](#)
- [4. Arquitectura general](#)
- [5. Tecnologías utilizadas](#)
- [6. Estructura del proyecto](#)
- [7. Módulos de aplicación y capa de servicios](#)
- [8. Comunicación entre componentes](#)
- [9. Configuración del entorno](#)
- [10. Variables de entorno necesarias](#)
- [11. Instalación paso a paso](#)
- [12. Configuración de base de datos](#)
- [13. Dependencias y librerías utilizadas](#)
- [14. Endpoints y rutas de la aplicación](#)
- [15. Flujo general del sistema](#)
- [16. Seguridad y autenticación](#)
- [17. Manejo de errores](#)

- 18. [Despliegue del sistema](#)
- 19. [Recomendaciones técnicas](#)
- 20. [Conclusiones](#)

1. Introducción del sistema

Vertice Muisca es una aplicación web desarrollada en **ASP.NET Core 8** que permite administrar operaciones típicas de un autoservicio: catálogo de productos y categorías, proveedores, entradas de inventario, consulta de stock, registro de ventas e informes consolidados.

La interfaz es **MVC con vistas Razor** servidas por el servidor. La persistencia se realiza en **SQLite** mediante **Entity Framework Core**. No existe, en el código analizado, una API REST independiente (proyecto Web API separado); las operaciones se exponen como **rutas MVC** y, en casos puntuales, respuestas **JSON** desde controladores.

Nota de nomenclatura: El ensamblado del proyecto se llama `VerticeMuiscaWeb` (typo histórico en `.csproj`), mientras que el espacio de nombres del código fuente es `VerticeMuiscaWeb`.

2. Objetivo general del proyecto

Centralizar en una única aplicación web las funciones de:

- Mantener el **catálogo** (categorías y productos).
- Gestionar **proveedores** y asociarlos a entradas de inventario.
- Registrar **movimientos de stock** (entradas positivas y salidas por ventas).
- Ejecutar **ventas** con detalle por producto y método de pago.
- Consultar **disponibilidad de stock** y alertas de mínimo.
- Generar **informes** de ventas, compras, catálogo y comparación de precios por proveedor.

El panel de inicio (`Dashboard`) actúa como punto de entrada operativo del sistema.

3. Alcance del sistema

3.1 Funcionalidades incluidas (según código)

Módulo	Funcionalidad
Dashboard	Resumen de contadores y accesos rápidos
Categorías	CRUD de categorías de producto
Productos	CRUD de productos (precio, código de barras, categoría, manejo de stock)
Proveedores	CRUD de proveedores; listado con soporte AJAX

Módulo	Funcionalidad
Inventario	Registro de entradas: producto, cantidad, stock mínimo, proveedor (HU6), precio de compra, margen %, precio de venta sugerido (cálculo en tiempo real) y opción de aplicar precio al catálogo
Consulta de stock	Stock agregado por producto y alertas
Ventas	Registro de venta e historial
Informes	Reportes por pestañas; impresión y PDF por sección activa o PDF completo (QuestPDF)

3.2 Fuera de alcance (no implementado en el código revisado)

- Autenticación de usuarios (login, roles, JWT, Identity).
- API REST documentada con OpenAPI/Swagger.
- Despliegue en contenedores (Docker) — no hay `Dockerfile` en el repositorio.
- Integración con pasarelas de pago externas.
- Microservicios desplegados de forma independiente.

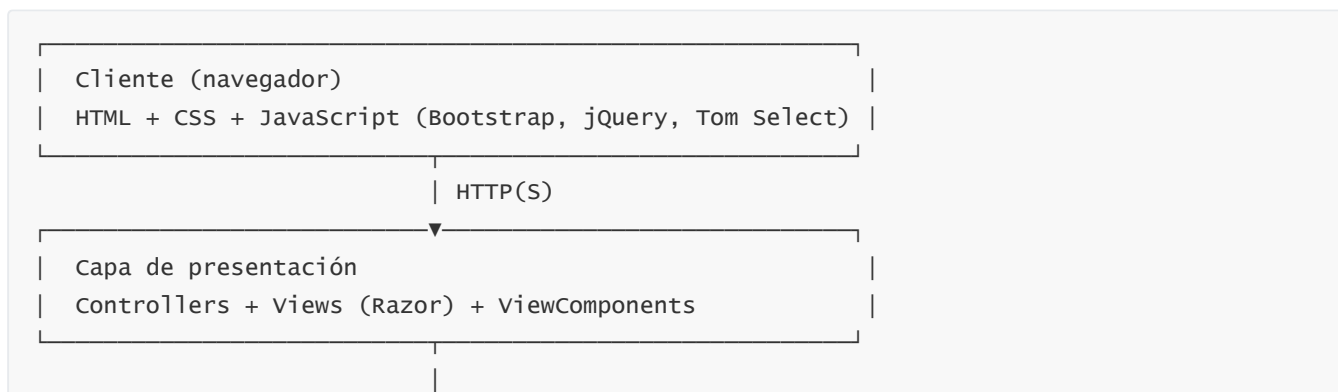
3.3 Artefactos adicionales en el repositorio

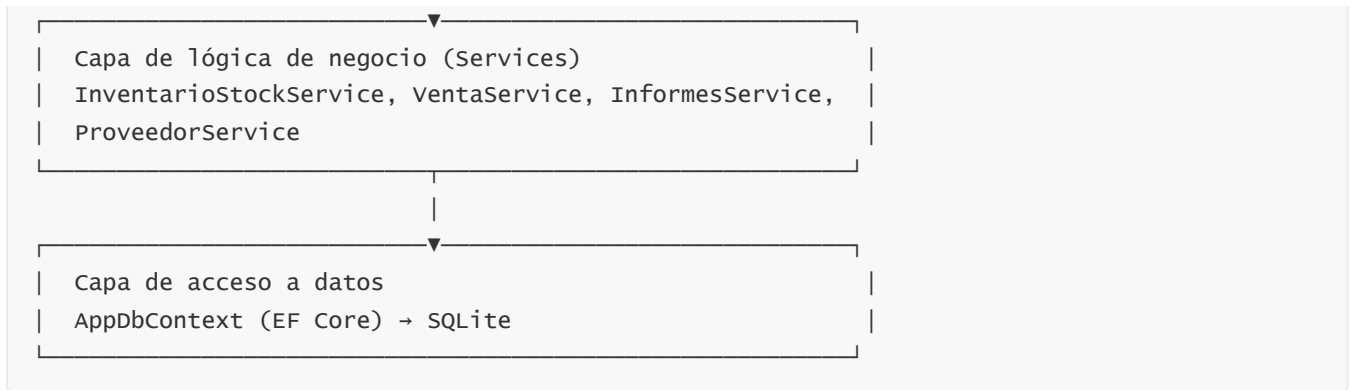
- `BasesDeDatos/DBinventario_s1y2.mwb`: modelo de datos MySQL Workbench (referencia de diseño; la app en ejecución usa SQLite). Tras la extensión de entrada de inventario, conviene alinear el `.mwb` con las columnas `idProveedor`, `precioUnitarioCompra`, `porcentajeMargenVenta` y `precioVentaSugerido` en `Stock` y la relación con `Proveedor` (documentadas en §12.3).
- Migraciones históricas que crearon tablas de **alertas por correo** (`ConfiguracionAlertaStock`). Esas entidades **no están** registradas en el `AppDbContext` actual; la tabla puede existir en bases de datos antiguas sin ser usada por la aplicación vigente.

4. Arquitectura general

4.1 Patrón arquitectónico

Arquitectura **monolítica en capas**:



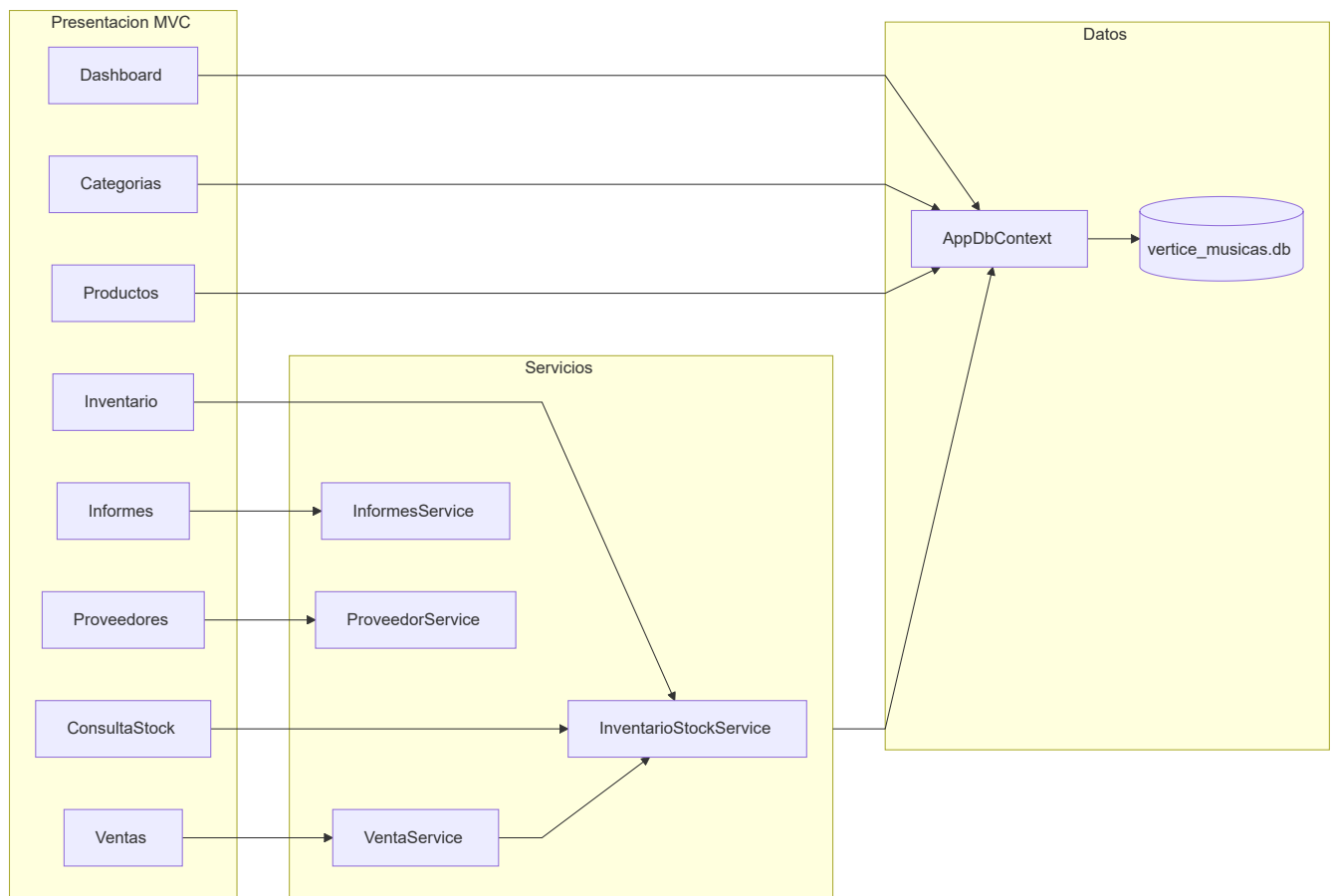


4.2 Punto de entrada

`Program.cs` configura:

- Inyección de dependencias (`AddDbContext`, servicios scoped).
- Pipeline HTTP: excepciones, HTTPS, archivos estáticos, enrutamiento, autorización.
- Migraciones EF al arranque + rutina `EnsureSprint3Schema` para compatibilidad de esquema.
- Ruta por defecto: `{controller=Dashboard}/{action=Index}/{id?}`.

4.3 Diagrama de módulos funcionales



5. Tecnologías utilizadas

Tecnología	Versión / detalle	Uso en el proyecto
.NET	8.0 (net8.0)	Runtime y SDK
ASP.NET Core MVC	8.x	Framework web
C#	12 (implícito en .NET 8)	Lenguaje backend
Razor Views	—	Vistas .cshtml
Entity Framework Core	8.*	ORM
SQLite	vía Microsoft.EntityFrameworkCore.Sqlite	Base de datos embebida
Bootstrap	5.x (wwwroot/lib/bootstrap)	UI responsive
jQuery	3.x	Validación y scripts
jQuery Validation Unobtrusive	—	Validación cliente
Tom Select	2.3.1 (CDN en vistas Inventario/Productos)	Selectores con búsqueda
JavaScript vanilla	—	Módulo Proveedores (proveedores.js)

Herramientas de desarrollo:

- Visual Studio / Visual Studio Code.
- dotnet CLI para compilación y EF Tools para migraciones.
- PowerShell: scripts Iniciar-Vertice-web.ps1, build-safe.ps1.

6. Estructura del proyecto

```
Ingenieria softw/  
├─ Ingenieria softw.sln          # Solución Visual Studio  
├─ BasesDeDatos/  
│   └─ DBinventario_sly2.mwb    # Modelo ER (referencia)  
└─ verticeMusicasWeb/  
    ├─ Program.cs               # Arranque, DI, migraciones  
    ├─ appsettings.json  
    ├─ appsettings.Development.json  
    ├─ verticeMusicasWeb.csproj  
    ├─ vertice_musicas.db        # Base SQLite (runtime)  
    ├─ controllers/             # Controladores MVC  
    ├─ Data/  
    │   └─ AppDbContext.cs      # Contexto EF Core
```

```

├─ Models/                # Entidades y ViewModels
├─ Services/              # Lógica de negocio
├─ ViewComponents/        # Componentes reutilizables UI
├─ Views/                 # Vistas Razor por módulo
├─ Migrations/            # Migraciones EF Core
├─ wwwroot/               # Estáticos (css, js, img)
├─ Properties/
|   └─ launchSettings.json
├─ Iniciar-Vertice-web.ps1
├─ Iniciar-Vertice-web.cmd
└─ build-safe.ps1

```

6.1 Controladores

Controlador	Responsabilidad
DashboardController	Página de inicio y KPIs
CategoriasController	CRUD categorías
ProductosController	CRUD productos
ProveedoresController	CRUD proveedores + endpoints JSON
InventarioController	Listado de movimientos; alta de entradas (proveedor, compra, margen, precio sugerido, aplicar precio al producto)
ConsultaStockController	Consulta de stock disponible
VentasController	Registro e historial de ventas
InformesController	Panel de informes
HomeController	Plantilla Privacy/Error (no es la página principal)

6.2 Servicios de aplicación

Servicio	Archivo
InventarioStockService	Services/InventarioStockService.cs
VentaService	Services/VentaService.cs
InformesService	Services/InformesService.cs
InformesPdfService	Services/InformesPdfService.cs
ProveedorService	Services/ProveedorService.cs

6.3 Registro en contenedor DI (Program.cs)

```
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlite($"Data Source={dbPath}"));
builder.Services.AddScoped<InventarioStockService>();
builder.Services.AddScoped<VentaService>();
builder.Services.AddScoped<InformesService>();
builder.Services.AddScoped<InformesPdfService>();
builder.Services.AddScoped<ProveedorService>();
```

7. Módulos de aplicación y capa de servicios

Aclaración importante: Este proyecto **no implementa microservicios**. No hay servicios desplegados por separado ni comunicación HTTP entre procesos backend. La sección describe los **módulos funcionales** y la **capa de servicios internos** del monolito.

7.1 InventarioStockService

Archivo: Services/InventarioStockService.cs

Función: Única capa autorizada para altas y bajas de stock (entradas de inventario y salidas por venta).

Método	Descripción
RegistrarEntradaInventarioAsync	Punto de entrada recomendado para el formulario de entrada: delega en RegistrarMovimientoAsync y devuelve EntradaInventarioResultado (incluye si se actualizó el precio del producto).
RegistrarMovimientoAsync	Crea cabecera Inventario + línea en tabla stock (MovimientoStock). Sobrecargas para ventas (sin proveedor/margen) y entradas completas.
CalcularPrecioVentaSugerido	Estático. Fórmula: $\text{precioCompra} + (\text{precioCompra} \times \text{porcentajeMargen} / 100)$, redondeo a 2 decimales.
ObtenerStockCantidadActualAsync	SUM(cantidad) por producto.
GetMovimientosAsync	Listado para /Inventario (filtros categoría, proveedor, texto).
GetStockDisponibleAsync	Stock agregado; enriquece último proveedor y precio de compra.
ObtenerStockMinimoActualAsync	MAX(stockMinimo) por producto.
ObtenerItemAlertaSiStockEnMinimoAsync	Alertas tras venta si stock $\leq$ mínimo.

Reglas de negocio — entradas (cantidad > 0):

Campo / regla	Validación
<code>idProveedor</code>	Obligatorio; debe existir en tabla <code>Proveedor</code> .
<code>precioUnitarioCompra</code>	Obligatorio; > 0 (no cero ni negativo).
<code>porcentajeMargenVenta</code>	Obligatorio; > 0 .
<code>precioVentaSugerido</code>	Obligatorio; > 0 ; valor enviado desde el formulario (calculado o ajustado por el administrador).
<code>aplicarPrecioAlProducto</code>	Si es <code>true</code> , actualiza <code>Producto.precio</code> con <code>precioVentaSugerido</code> en la misma transacción.

Reglas de negocio — salidas (`cantidad < 0`): usadas por `VentaService`; `idProveedor`, precios y margen quedan en `null`.

Transacciones: `beginTransactionAsync` cuando no hay transacción activa (p. ej. venta ya abre una).

7.1.1 Módulo de entrada de inventario (Sprint 2 / HU6)

Capa	Archivo	Responsabilidad
Vista GET/POST	<code>Views/Inventario/Create.cshtml</code>	Formulario, Tom Select, cálculo JS del precio sugerido, checkbox aplicar precio.
Controlador	<code>Controllers/InventarioController.cs</code>	Validación <code>ModelState</code> , prerequisites (productos y proveedores), <code>TempData</code> .
ViewModel	<code>Models/RegistrarEntradaViewModel.cs</code>	Campos del POST.
Resultado	<code>Models/EntradaInventarioResultado.cs</code>	Confirmación de actualización de precio del producto.
Listado	<code>Views/Inventario/Index.cshtml</code>	Columnas: proveedor, P. compra, margen %, P. venta sug.

Prerrequisitos en GET `/Inventario/Create`: al menos un producto y un proveedor; si faltan, redirección con `TempData["Error"]`.

7.2 VentaService

Función: Búsqueda de productos para venta y registro de ventas con impacto en stock.

- Valida líneas de venta, método de pago y stock disponible cuando el producto `ManejaStock`.
- Persiste `Venta` + `DetalleVenta`.
- Genera movimiento de stock negativo vía `InventarioStockService`.
- Devuelve `ventaRegistroResultado` con posibles alertas de stock mínimo.

7.3 ProveedorService

Función: CRUD lógico de proveedores.

- Validación de nombre y contacto obligatorios.
- Unicidad de nombre (comparación sin distinguir mayúsculas).
- Retorna `ProveedorOperacionResultado` para vistas y AJAX.

7.4 InformesService

Función: Construcción de `InformesViewModel` con agregaciones en memoria y consultas EF.

Incluye: ventas, entradas/salidas de inventario, compras por proveedor, comparación de precios entre proveedores, catálogo de productos, resumen por categoría y ranking de más vendidos.

Limitación técnica documentada: Algunas agregaciones de precios (`Min`/`Max` sobre `decimal`) se resuelven en memoria tras `ToListAsync()` por restricciones del proveedor SQLite en EF Core.

7.4.1 InformesPdfService y exportación por sección

Archivo: `Services/InformesPdfService.cs`

Paquete: QuestPDF (licencia Community).

Método	Descripción
<code>GenerarPdf(InformesViewModel vm, string seccion)</code>	Genera PDF solo de la sección indicada
Secciones en <code>InformesSeccion</code>	Constantes: <code>ventas</code> , <code>mas-vendidos</code> , <code>salidas</code> , <code>entradas</code> , <code>compras-proveedor</code> , <code>comparacion</code> , <code>proveedores</code> , <code>productos</code> , <code>categorias</code> , <code>completo</code>

Regla importante: Si el parámetro `seccion` falta o es inválido, `InformesSeccion.Normalizar()` usa `ventas` (nunca exporta todo por defecto). Solo `seccion=completo` genera el informe integral.

Vista `Informes/Index.cshtml`:

Control	Comportamiento
Imprimir	Imprime únicamente la pestaña visible (oculta el resto con CSS + <code>display:none</code> en JS)
Exportar PDF	Envía formulario GET oculto <code>#formExportPdf</code> con <code>seccion</code> = pestaña activa
PDF completo	Mismo formulario con <code>seccion=completo</code>

La UI tiene **tres grupos** de pestañas (Ventas, Compras, Catálogo). Bootstrap solo quita `active` dentro del mismo `<ul>`, por lo que el JS mantiene una única sección en variables (`seccionActiva`) y sincroniza manualmente nav-links y `.tab-pane` al cambiar de pestaña o al imprimir/exportar.

7.5 Controladores sin servicio dedicado

`CategoriasController` y `ProductosController` acceden **directamente** a `AppDbContext` (patrón CRUD simple).

8. Comunicación entre componentes

8.1 No hay comunicación entre microservicios

Toda la comunicación backend es **in-process** dentro del mismo proceso Kestrel:

Origen	Destino	Mecanismo
Controller	Service	Inyección de dependencias (constructor)
VentaService	InventarioStockService	DI
Service	AppDbContext	DI
View	ViewComponent	Invocación Razor <code>@await Component.InvokeAsync</code>
Vista Proveedores	ProveedoresController	<code>fetch</code> HTTP + header <code>X-Requested-With: XMLHttpRequest</code>

8.2 Flujo HTTP típico (MVC)

1. Petición GET/POST a ruta `/Controlador/Accion`.
2. Model binding de parámetros y formularios.
3. Ejecución de acción en controlador o servicio.
4. Respuesta: `View()`, `RedirectToAction()`, o `Json()`.

8.3 Ejemplo: registro AJAX de proveedor

```
POST /Proveedores/Create
Content-Type: application/x-www-form-urlencoded
X-Requested-With: XMLHttpRequest

__RequestVerificationToken=...&IdProveedor=0&Nombre=...&Contacto=...
```

Respuesta JSON (`ProveedorOperacionResultado`):

```
{
  "exito": true,
  "mensaje": "Proveedor registrado correctamente.",
  "proveedor": { "idProveedor": 1, "nombre": "...", "contacto": "..." },
  "errores": {}
}
```

8.4 ViewComponent global

`AlertaStockViewComponent` se invoca desde `_Layout.cshtml` en cada página y consulta `InventarioStockService` para productos bajo stock mínimo.

8.5 POST entrada de inventario (/Inventario/Create)

No es un endpoint REST JSON; es formulario MVC con antiforgery.

Campos del body (`application/x-www-form-urlencoded`):

Campo	Tipo	Notas
<code>Fecha</code>	<code>DateTime</code>	Fecha del movimiento
<code>IdProducto</code>	<code>int</code>	Producto existente
<code>Cantidad</code>	<code>int</code>	> 0
<code>StockMinimo</code>	<code>int</code>	≥ 0
<code>IdProveedor</code>	<code>int</code>	Proveedor HU6
<code>PrecioUnitarioCompra</code>	<code>decimal</code>	> 0
<code>PorcentajeMargenVenta</code>	<code>decimal</code>	> 0; default 30 en GET
<code>PrecioVentaSugerido</code>	<code>decimal</code>	> 0; editable en UI
<code>AplicarPrecioAlProducto</code>	<code>bool</code>	Checkbox; actualiza <code>Producto.precio</code> si marcado

Respuesta exitosa: redirect a `/Inventario` (PRG) con:

- `TempData["Success"]` — entrada registrada.
- `TempData["PrecioProductoActualizado"]` — solo si se aplicó precio al producto.

Flujo interno: `InventarioController.Create(POST)` → `RegistrarEntradaInventarioAsync` → `RegistrarMovimientoAsync`.

9. Configuración del entorno

9.1 Requisitos mínimos

Requisito	Versión recomendada
Sistema operativo	Windows 10/11 (scripts PowerShell incluidos); compatible con Linux/macOS vía <code>dotnet</code>
.NET SDK	8.0.x
Navegador	Chrome, Edge o Firefox actualizado

Requisito	Versión recomendada
Espacio en disco	~500 MB (SDK + dependencias + base de datos)

9.2 Perfiles de ejecución (launchSettings.json)

Perfil	URL
http	http://127.0.0.1:0 (puerto dinámico)
https	https://127.0.0.1:0 y http://127.0.0.1:0
UnClíc	http://127.0.0.1:5288
IIS Express	http://localhost:60170

9.3 Resolución de ruta de base de datos

Program.cs define ResolveDatabasePath() para mantener un único archivo canónico:

```
{ContentRootPath}/vertice_musicas.db
```

Con lógica de copia desde directorio de ejecución o bin si el archivo canónico no existe o está desactualizado.

10. Variables de entorno necesarias

Variable	Obligatoria	Valor típico	Uso
ASPNETCORE_ENVIRONMENT	Recomendada	Development / Production	Perfil de configuración
ASPNETCORE_URLS	Opcional	http://127.0.0.1:5288	URL de escucha (script Iniciar-vertice-web.ps1)

10.1 Configuración en archivos JSON

appsettings.json:

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Data Source=vertice_musicas.db"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

Nota: La cadena `DefaultConnection` en `appsettings.json` **no es utilizada** directamente en `Program.cs`; la conexión SQLite se construye en código con `ResolveDatabasePath()`. Si se desea centralizar la ruta, habría que refactorizar `Program.cs` (no implementado actualmente).

`appsettings.Development.json`: Solo ajusta niveles de logging.

No se detectaron variables de entorno para secretos, API keys activas en el `DbContext` actual, ni configuración de correo en uso en el código de servicios vigente.

11. Instalación paso a paso

11.1 Obtener el código

```
cd "C:\Users\ediso\OneDrive\Escritorio\Ingenieria softw"
```

(Ajustar ruta según ubicación real del clon o copia del proyecto.)

11.2 Verificar SDK

```
dotnet --version
```

Debe reportar SDK 8.0.x.

11.3 Restaurar y compilar

```
cd VerticeMusicasWeb
dotnet restore
dotnet build
```

Alternativa con limpieza de procesos bloqueados:

```
.\build-safe.ps1
# o con clean:
.\build-safe.ps1 -clean
```

11.4 Ejecutar la aplicación

Opción A — dotnet CLI:

```
dotnet run
```

Opción B — Script con puerto fijo y navegador:

```
.\Iniciar-Vertice-Web.ps1
# o puerto personalizado:
.\Iniciar-Vertice-Web.ps1 -Puerto 5290
```

Opción C — Perfil UnClic:

```
dotnet run --launch-profile UnClic
```

11.5 Acceder al sistema

Abrir en el navegador la URL mostrada en consola (ej. `http://127.0.0.1:5288`). La ruta por defecto carga `Dashboard/Index`.

12. Configuración de base de datos

12.1 Motor y archivo

Parámetro	Valor
Motor	SQLite
Archivo	<code>VerticeMusicasweb/vertice_musicas.db</code>
ORM	Entity Framework Core 8

12.2 Modelo de datos (entidades activas en `AppDbContext`)

Tabla BD	Entidad C#	Clave primaria
<code>Categoria</code>	<code>Categoria</code>	<code>idCategoria</code>
<code>Proveedor</code>	<code>Proveedor</code>	<code>idProveedor</code> (índice único en <code>nombre</code>)
<code>Producto</code>	<code>Producto</code>	<code>idProducto</code> (índice único en <code>codigoBarras</code>)
<code>Inventario</code>	<code>Inventario</code>	<code>idInventario</code>
<code>Stock</code>	<code>MovimientoStock</code>	<code>idStock</code>
<code>Venta</code>	<code>Venta</code>	<code>idVenta</code>
<code>DetalleVenta</code>	<code>DetalleVenta</code>	<code>idDetalle</code>

12.3 Relaciones principales

```
Categoria 1 — * Producto
Producto 1 — * MovimientoStock (tabla Stock)
Inventario 1 — * MovimientoStock
Proveedor 1 — * MovimientoStock (opcional, FK nullable en entradas)
Venta 1 — * DetalleVenta
Producto 1 — * DetalleVenta
```

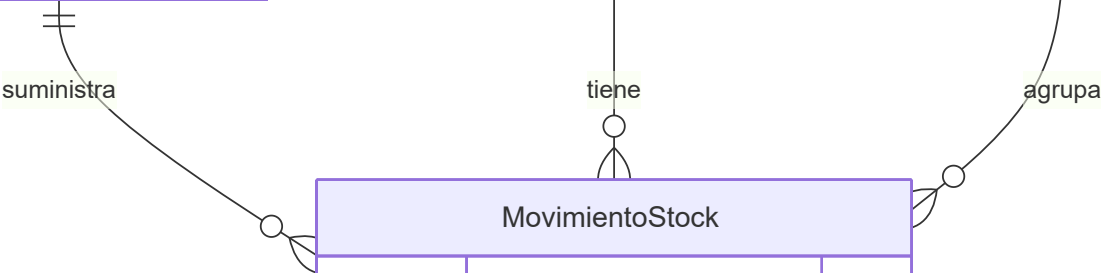
Relación `MovimientoStock` — `Proveedor`: cada entrada de inventario (`cantidad > 0`) registra el proveedor de la mercancía y el precio de compra en el movimiento. Las salidas por venta (`cantidad < 0`) dejan `idProveedor`, `precioUnitarioCompra`, `porcentajeMargenVenta` y `precioVentaSugerido` en null.

Proveedor		
int	idProveedor	PK
string	nombre	
string	contacto	

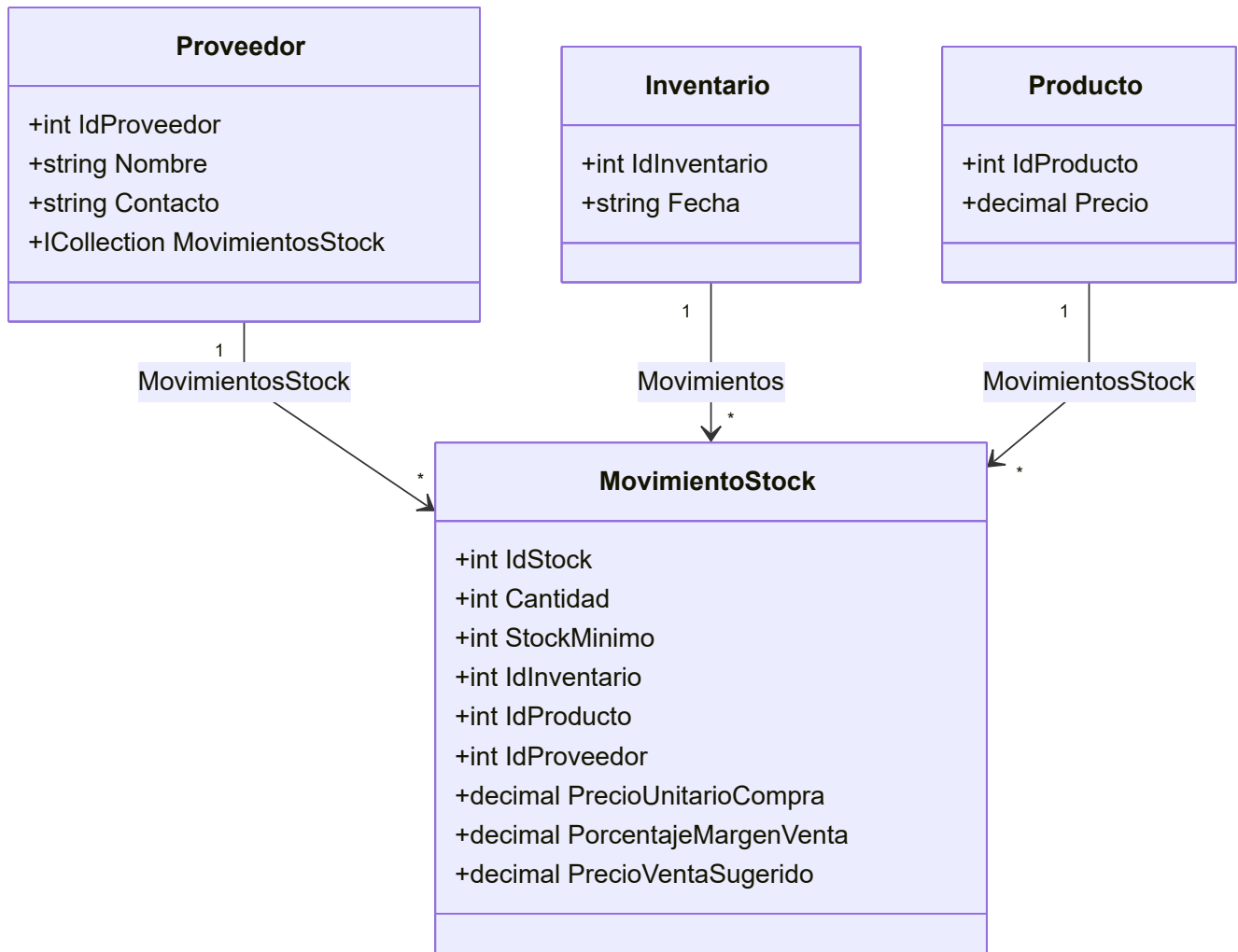
Producto

Inventario

MovimientoStock		
int	idStock	PK
int	cantidad	
int	idProveedor	FK
decimal	precioUnitarioCompra	
decimal	porcentajeMargenVenta	
decimal	precioVentaSugerido	



12.3.1 Diagrama de clases (entidades de inventario)



12.4 Columnas relevantes en Stock (movimientos)

Columna	Tipo lógico	Descripción
<code>cantidad</code>	int	Positiva = entrada; negativa = salida (venta)
<code>stockMinimo</code>	int	Umbral registrado en el movimiento
<code>idProveedor</code>	int?	Proveedor en entradas (FK → <code>Proveedor</code>)
<code>precioUnitarioCompra</code>	decimal?	Precio de compra unitario en entradas
<code>porcentajeMargenVenta</code>	decimal?	Margen % aplicado en la entrada
<code>precioventaSugerido</code>	decimal?	Precio sugerido registrado: compra + (compra × margen / 100), ajustable por el administrador
<code>fecha</code> (en Inventario)	TEXT	Formato <code>yyyy-MM-dd</code> en entradas de inventario

12.5 Migraciones EF Core

Aplicación automática al iniciar: `db.Database.Migrate()` en `Program.cs`.

Migración	Descripción
<code>20260403034442_SchemaCompletoInventario</code>	Esquema base
<code>20260428010000_Sprint3VentasInventario</code>	Ventas y ajustes de stock
<code>20260512123620_ConfiguracionAlertaStock</code>	Tabla configuración alertas (legado)
<code>20260512125319_ClaveApiWhatsAppColumn</code>	Columna WhatsApp (legado)
<code>20260512125840_AlertasSoloCorreo</code>	Ajuste alertas (legado)
<code>20260512130712_ConfiguracionAlertaStockGmailCols</code>	Columnas Gmail (legado)
<code>20260525022731_AgregarTablaProveedor</code>	Tabla Proveedor
<code>20260525024254_ProveedorEnMovimientoStock</code>	FK proveedor y precio en Stock
<code>20260525175230_MargenYPrecioVentaEnMovimientoStock</code>	Margen % y precio de venta sugerido en Stock

Comandos manuales (desarrollo):

```
cd VerticeMusicasWeb
dotnet ef migrations add NombreMigracion --context AppDbContext
dotnet ef database update --context AppDbContext
```

12.6 Rutina adicional `EnsureSprint3Schema`

Ejecutada tras migraciones; crea tablas/columnas de ventas si faltan (compatibilidad con bases antiguas). No sustituye las migraciones EF para entornos nuevos.

13. Dependencias y librerías utilizadas

13.1 Paquetes NuGet (`VerticeMusicasWeb.csproj`)

Paquete	Versión	Propósito
<code>Microsoft.EntityFrameworkCore.Sqlite</code>	8.*	Proveedor SQLite
<code>Microsoft.EntityFrameworkCore.Tools</code>	8.*	Migraciones (CLI)
<code>Microsoft.AspNetCore.Mvc.Razor.RuntimeCompilation</code>	8.*	Recompilación de vistas en Development
<code>QuestPDF</code>	2026.*	Generación de PDF en módulo Informes

13.2 Dependencias frontend (wwwroot)

Librería	Ubicación
Bootstrap 5	wwwroot/lib/bootstrap
jQuery	wwwroot/lib/jquery
jQuery Validation	wwwroot/lib/jquery-validation
jQuery Validation Unobtrusive	wwwroot/lib/jquery-validation-unobtrusive

13.3 Hojas de estilo propias

Archivo	Uso
wwwroot/css/site.css	Estilos globales
wwwroot/css/dashboard.css	Página de inicio
wwwroot/css/informes.css	Módulo informes
wwwroot/css/proveedores.css	Módulo proveedores

14. Endpoints y rutas de la aplicación

Convención: {controller=Dashboard}/{action=Index}/{id?} — rutas sin prefijo /api.

14.1 Rutas principales (GET)

Ruta	Controlador	Descripción
/ o /Dashboard	Dashboard	Inicio
/Categorias	Categorias	Listado
/Categorias/Create	Categorias	Formulario alta
/Categorias/Edit/{id}	Categorias	Formulario edición
/Productos	Productos	Listado (?categoriaId=&q=)
/Productos/Create	Productos	Alta
/Productos/Edit/{id}	Productos	Edición
/Proveedores	Proveedores	Listado (?q=)
/Proveedores/Create	Proveedores	Alta
/Proveedores/Edit/{id}	Proveedores	Edición
/Inventario	Inventario	Movimientos (?categoriaId=&proveedorId=&q=)
/Inventario/Create	Inventario	Nueva entrada
/ConsultaStock	ConsultaStock	Stock (?categoriaId=&proveedorId=&q=)
/Ventas/Create	Ventas	Registrar venta
/Ventas/Historial	Ventas	Historial

Ruta	Controlador	Descripción
/Informes	Informes	Informes (?desde=&hasta=&idProductoComparar=&idProveedor=&idCategoria=&q=)
/Informes/ExportPdf	Informes	Descarga PDF (?seccion=productos + mismos filtros)
/Home/Privacy	Home	Página plantilla
/Home/Error	Home	Error (producción)

14.2 Rutas POST (formularios MVC)

Todas las operaciones de escritura relevantes usan `[ValidateAntiForgeryToken]` en POST (protección CSRF en formularios).

Ruta POST	Acción
/Categorias/Create	Crear categoría
/Categorias/Edit/{id}	Actualizar categoría
/Productos/Create	Crear producto
/Productos/Edit/{id}	Actualizar producto
/Proveedores/Create	Crear proveedor
/Proveedores/Edit/{id}	Actualizar proveedor
/Inventario/Create	Registrar entrada inventario (proveedor, compra, margen, precio sugerido, aplicar al producto)
/Ventas/Create	Registrar venta

14.3 Endpoints JSON (no API REST formal)

Método	Ruta	Respuesta	Condición
GET	/Proveedores/Listar?q=	application/json lista proveedores	—
GET	/Proveedores/Obtener/{id}	JSON proveedor o 404	—
POST	/Proveedores/Create	JSON ProveedorOperacionResultado	Header X-Requested-With: XMLHttpRequest
POST	/Proveedores/Edit/{id}	JSON ProveedorOperacionResultado	AJAX

No hay documentación Swagger ni controladores `[ApiController]` en el proyecto.

15. Flujo general del sistema

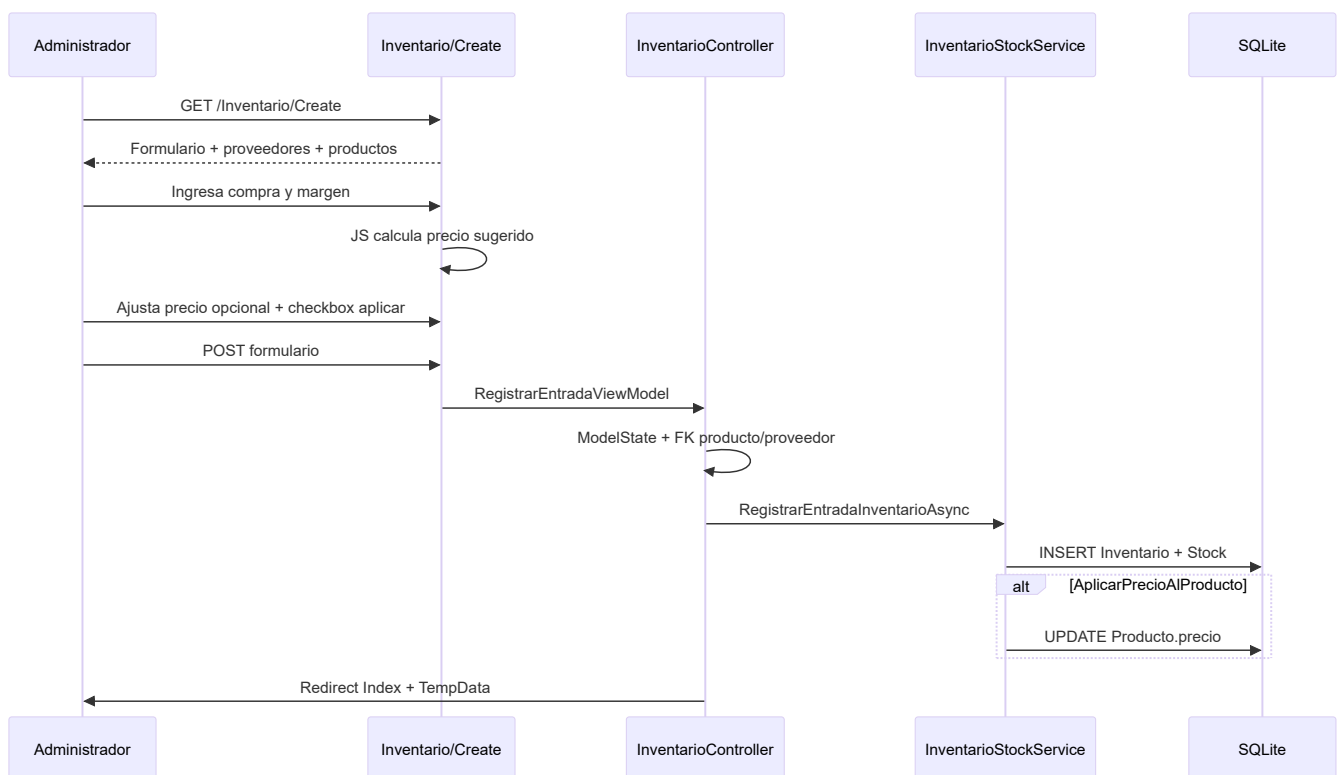
15.1 Flujo de arranque

1. `webApplication.CreateBuilder`.
2. Registro de MVC, DbContext y servicios.
3. `Migrate()` + `EnsureSprint3Schema()`.
4. Pipeline middleware y escucha HTTP.

15.2 Flujo operativo recomendado (datos maestros)

Categorías → Productos → Proveedores → Entrada inventario → Ventas → Informes / Consulta stock

15.3 Flujo de entrada de inventario



Pasos detallados:

1. El administrador abre `/Inventario/Create` (requiere productos y proveedores previos).
2. Selecciona **proveedor** (catálogo HU6), **producto**, cantidad, stock mínimo, **precio de compra** y **porcentaje de margen** (ambos > 0).
3. El navegador calcula en tiempo real el **precio de venta sugerido**:
$$\text{precioCompra} + (\text{precioCompra} \times \text{porcentaje} / 100)$$

(equivalente a `CalcularPrecioVentaSugerido` en servidor).
4. Puede **ajustar** el precio sugerido manualmente o pulsar **Recalcular**.
5. Opcional: marca **Aplicar este precio de venta al producto** para actualizar `Producto.precio` al

guardar.

6. `InventarioController` valida y ejecuta `RegistrarEntradaInventarioAsync`.
7. Persistencia en `Stock: idProveedor, precioUnitarioCompra, porcentajeMargenVenta, precioVentaSugerido` (valor final del formulario).
8. Redirección a `/Inventario` con mensajes en `TempData`.

Criterios de aceptación cubiertos:

Criterio	Cumplimiento
Seleccionar proveedor de la entrada	<code>IdProveedor</code> + Tom Select
Precio de compra y margen > 0	Data Annotations + validación en servicio
Cálculo automático del precio sugerido	JavaScript en <code>Create.cshtml</code>
Confirmar o ajustar precio antes de aplicar	Campo editable + checkbox <code>AplicarPrecioAlProducto</code>
Datos en el movimiento	Columnas en tabla <code>Stock</code>
Confirmación de precio de producto actualizado	<code>TempData["PrecioProductoActualizado"]</code> en <code>_Layout</code>

15.4 Flujo de venta

1. Usuario en `/Ventas/Create` agrega líneas de productos.
2. `VentaService.RegistrarVentaAsync` valida stock y precios.
3. Transacción: cabecera `venta`, detalles, movimiento stock **negativo**.
4. Opcional: alertas si stock queda en o bajo mínimo.

15.5 Cálculo de stock disponible

Stock actual por producto = **suma** de `cantidad` en tabla `Stock` para ese `idProducto`.

15.6 Exportación e impresión de informes (por sección)

1. Usuario aplica filtros en `/Informes` (GET) y abre una pestaña (Ventas, Productos, Entradas, etc.).
2. La barra **Sección visible** muestra el nombre del informe activo.
3. **Imprimir:** JavaScript oculta todas las `.informes-print-section` excepto la activa y llama a `window.print()`.
4. **Exportar PDF:** el botón asigna `seccion` en el formulario oculto y hace GET a `/Informes/ExportPdf`.
5. `InformesController.ExportPdf` normaliza `seccion` con `InformesSeccion.Normalizar()` y llama a `InformesPdfService.GenerarPdf`.
6. El archivo descargado se nombra `Informe_{seccion}_{fecha}.pdf` (ej. `Informe_productos_20260525_1430.pdf`).

Valores de `seccion`:

Pestaña en UI	Parámetro <code>seccion</code>
Ventas — Listado	<code>ventas</code>
Más vendidos	<code>mas-vendidos</code>
Salidas	<code>salidas</code>
Entradas	<code>entradas</code>
Compras por proveedor	<code>compras-proveedor</code>
Comparar precios	<code>comparacion</code>
Resumen proveedores	<code>proveedores</code>
Catálogo productos	<code>productos</code>
Categorías	<code>categorias</code>
Botón «PDF completo»	<code>completo</code>

Problema conocido (corregido): si varios `.nav-link` quedaban con `active` a la vez, la impresión podía mostrar varias tablas y el PDF podía tomar la sección equivocada (la primera del DOM). La vista sincroniza una sola pestaña activa antes de imprimir o enviar el formulario PDF.

16. Seguridad y autenticación

16.1 Estado actual (según código)

Aspecto	Estado
Autenticación de usuarios	No implementada
Autorización por roles	No implementada
<code>UseAuthorization()</code>	Registrado en pipeline sin políticas configuradas
Acceso a la aplicación	Público para quien alcance la URL del servidor

16.2 Medidas presentes

Medida	Implementación
Anti-forgery	<code>[ValidateAntiForgeryToken]</code> en POST de formularios
HTTPS	Redirección habilitada (<code>useHttpsRedirection</code>)
HSTS	Habilitado fuera de Development
Validación de entrada	Data Annotations en modelos + <code>ModelState</code>

16.3 Consideraciones

- La aplicación está orientada a entorno controlado (LAN o equipo local).
- Para producción expuesta a Internet se recomienda implementar autenticación (ASP.NET Core Identity o similar) — **no incluida en la versión analizada**.

17. Manejo de errores

17.1 Errores de validación (UI)

- `ModelState` en controladores CRUD.
- `TempData["Error"]`, `TempData["Success"]`, `TempData["Warning"]`, `TempData["PrecioProductoActualizado"]` mostrados en `_Layout.cshtml`.
- `ProveedorOperacionResultado` con diccionario `Errores` para AJAX.

17.2 Excepciones en servicios

Origen	Comportamiento
<code>InventarioStockService</code>	<code>InvalidOperationException</code> , <code>ArgumentOutOfRangeException</code>
<code>VentaService</code>	<code>InvalidOperationException</code> en reglas de negocio
Controladores	<code>try/catch</code> en Create de Inventario/Productos; mensaje en TempData

Ejemplo en inventario:

```
catch (Exception ex)
{
    TempData["Error"] = $"No se pudo registrar el movimiento: {ex.Message}";
    // ...
}
```

17.3 Errores no controlados

- **Development:** página de excepciones del desarrollador (detalle completo).
- **Production:** redirección a `/Home/Error` con `ErrorViewModel` (`RequestId` / trace identifier).

17.4 Errores de compilación de vistas

Con `AddRazorRuntimeCompilation()` en Development, errores de sintaxis Razor (ej. operadores no soportados) producen HTTP 500 hasta corregir la vista.

17.5 Logging

Configurado vía `appsettings*.json`:

- Default: `Information`
- `Microsoft.AspNetCore`: `Warning`

No hay sink externo (Archivo, Application Insights, etc.) configurado en el repositorio.

18. Despliegue del sistema

18.1 Publicación para producción

```
cd VerticeMusicasWeb
dotnet publish -c Release -o ./publish
```

El folder `publish` contendrá:

- `VerticeMuiscaWeb.dll` y dependencias.
- `wwwroot` (estáticos).
- `appsettings.json`.
- Es necesario incluir o crear `vertice_musicas.db` en la ruta esperada por `ResolveDatabasePath` (junto al contenido publicado o en `ContentRootPath`).

18.2 Ejecución publicada

```
cd publish
$env:ASPNETCORE_ENVIRONMENT = "Production"
dotnet VerticeMuiscaWeb.dll --urls "http://0.0.0.0:5000"
```

18.3 IIS / Windows

No hay configuración `web.config` generada en el análisis del repo; puede crearse con:

```
dotnet publish -c Release
```

e instalar el **Hosting Bundle** de ASP.NET Core 8 en el servidor Windows.

18.4 Linux

Ejecución self-contained o framework-dependent con Kestrel detrás de Nginx como reverse proxy (configuración externa al repo).

18.5 Base de datos en despliegue

- Respalidar `vertice_musicas.db` periódicamente (copia de archivo).
- Las migraciones se aplican al arrancar la app; verificar permisos de escritura en el directorio de la BD.

No documentado en el repo: CI/CD, Docker, Kubernetes ni scripts de despliegue automatizado.

19. Recomendaciones técnicas

1. **Autenticación:** Implementar Identity o JWT si el sistema sale del entorno local.
 2. **Cadena de conexión:** Usar `ConnectionStrings:DefaultConnection` desde configuración en lugar de solo `ResolveDatabasePath()` hardcodeado.
 3. **Unificar nombres:** Alinear `VerticeMuiscaWeb` / `VerticeMusicasWeb` en ensamblado y namespaces.
 4. **Legado BD:** Evaluar si tablas `ConfiguracionAlertaStock` deben eliminarse o reintegrarse al modelo.
 5. **API REST:** Si se requiere integración externa, agregar proyecto `Web API` o minimal APIs con Swagger.
 6. **Pruebas:** No hay proyecto de tests en la solución; añadir pruebas unitarias para servicios críticos (`VentaService`, `InventarioStockService`).
 7. **Proceso bloqueado en Windows:** Usar `build-safe.ps1` antes de compilar si `VerticeMuiscaWeb.exe` queda en ejecución.
 8. **SQLite en producción:** Para alta concurrencia, valorar migración a SQL Server o PostgreSQL.
 9. **Variables de entorno:** Externalizar URL y ruta de BD para despliegues multi-entorno.
-

20. Conclusiones

Vertice Muisca (VerticeMusicasWeb) es una aplicación monolítica ASP.NET Core 8 bien segmentada en controladores, servicios y capa de datos EF Core sobre SQLite. Cubre el ciclo operativo de un autoservicio: catálogo, proveedores, inventario, ventas e informes.

Puntos fuertes del diseño actual:

- Separación de lógica compleja en servicios (`InventarioStockService`, `VentaService`, `InformesService`, `ProveedorService`).
- Migraciones EF y parche de esquema para compatibilidad.
- UI modular por vistas con componentes reutilizables (alertas de stock).

Limitaciones documentadas:

- No es arquitectura de microservicios.
- Sin autenticación ni autorización de usuarios.
- Sin API REST formal; integración externa limitada a endpoints JSON puntuales.
- Configuración de conexión parcialmente duplicada entre `appsettings` y código en `Program.cs`.

Este manual refleja el estado del código en el repositorio analizado. Ante cambios futuros en el proyecto, debe actualizarse especialmente las secciones de migraciones, rutas y alcance funcional.

Referencias rápidas

Recurso	Ruta
Solución	Ingenieria softw.sln
Proyecto web	VerticeMusicasweb/VerticeMusicasweb.csproj
Arranque	VerticeMusicasweb/Program.cs
Contexto datos	VerticeMusicasweb/Data/AppDbContext.cs
Layout UI	VerticeMusicasweb/Views/Shared/_Layout.cshtml
Script inicio rápido	VerticeMusicasweb/Iniciar-Vertice-web.ps1

Documento generado a partir del análisis estático del repositorio. No sustituye documentación de usuario final ni manual de instalación para usuarios no técnicos.